KON426E – Intelligent Control Systems
Assignment 1

Atakan Kuşçu
Student ID: 040210520

April 1, 2026

1 Question 1: Nonlinear Regression

In this question, the objective is to estimate the parameters of a noisy scalar signal by fitting a nonlinear sinusoidal model to the given data. The signal is assumed to be approximately represented by

$$\hat{y}(t;\theta) = A\sin(\omega t + \phi) + c, \tag{1}$$

where the unknown parameter vector is

$$\theta = \begin{bmatrix} A & \omega & \phi & c \end{bmatrix}^T. \tag{2}$$

The estimation task was performed using three optimization methods:

1. Gradient Descent,
2. Gradient Descent with Momentum,
3. Newton's Method.

The performance of these methods was compared in terms of convergence behavior, estimated parameters, and test-set prediction accuracy.

2 Problem Formulation and Data Preparation

The MATLAB data file `signal.mat` contains the sampled time vector and the corresponding noisy signal values. Let the measured dataset be denoted by

$$\{(t_i, y_i)\}_{i=1}^{N}, \tag{3}$$

where t_i is the sampling instant and y_i is the measured signal value.

Following the homework instruction, the dataset was divided into training and test sets as follows:

- the first 80% of the samples were used for training,
- the last 20% of the samples were used for testing.

Parameter estimation was performed using only the training set. The test set was used only to evaluate the predictive performance of the fitted model.

Let N_{tr} denote the number of training samples. The model output for the i-th training sample is

$$\hat{y}_i = A\sin(\omega t_i + \phi) + c. \tag{4}$$

The error for each training sample is defined as

$$e_i(\theta) = y_i - \hat{y}_i = y_i - (A\sin(\omega t_i + \phi) + c)\,. \tag{5}$$

The training cost function was chosen as the sum of squared errors:

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{N_{\text{tr}}} e_i^2 = \frac{1}{2}\sum_{i=1}^{N_{\text{tr}}} \left[y_i - (A\sin(\omega t_i + \phi) + c)\right]^2. \tag{6}$$

This cost function is appropriate because it directly measures the mismatch between the measured noisy signal and the nonlinear sinusoidal model over the training set, and it leads to a standard nonlinear least-squares optimization problem.

3 Gradient and Hessian Derivation

3.1 Gradient of the Cost Function

Starting from

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{N_{\text{tr}}} e_i^2, \tag{7}$$

the gradient can be derived by applying the chain rule:

$$\frac{\partial J}{\partial \theta_j} = \sum_{i=1}^{N_{\text{tr}}} e_i \frac{\partial e_i}{\partial \theta_j}. \tag{8}$$

Since

$$e_i = y_i - \hat{y}_i, \tag{9}$$

we have

$$\frac{\partial e_i}{\partial \theta_j} = -\frac{\partial \hat{y}_i}{\partial \theta_j}. \tag{10}$$

Therefore,

$$\frac{\partial J}{\partial \theta_j} = -\sum_{i=1}^{N_{\text{tr}}} e_i \frac{\partial \hat{y}_i}{\partial \theta_j}. \tag{11}$$

Let

$$s_i = \sin(\omega t_i + \phi), \qquad c_i = \cos(\omega t_i + \phi). \tag{12}$$

Then the model becomes

$$\hat{y}_i = A s_i + c. \tag{13}$$

The first derivatives of the model are:

$$\frac{\partial \hat{y}_i}{\partial A} = s_i, \tag{14}$$

$$\frac{\partial \hat{y}_i}{\partial \omega} = A c_i t_i, \tag{15}$$

$$\frac{\partial \hat{y}_i}{\partial \phi} = A c_i, \tag{16}$$

$$\frac{\partial \hat{y}_i}{\partial c} = 1. \tag{17}$$

Hence, the gradient components are

$$\frac{\partial J}{\partial A} = -\sum_{i=1}^{N_{\text{tr}}} e_i s_i, \tag{18}$$

$$\frac{\partial J}{\partial \omega} = -\sum_{i=1}^{N_{\text{tr}}} e_i \, A c_i t_i, \tag{19}$$

$$\frac{\partial J}{\partial \phi} = -\sum_{i=1}^{N_{\text{tr}}} e_i \, A c_i, \tag{20}$$

$$\frac{\partial J}{\partial c} = -\sum_{i=1}^{N_{\text{tr}}} e_i. \tag{21}$$

Thus, the gradient vector is

$$\nabla J(\theta) = \begin{bmatrix} \dfrac{\partial J}{\partial A} \\ \dfrac{\partial J}{\partial \omega} \\ \dfrac{\partial J}{\partial \phi} \\ \dfrac{\partial J}{\partial c} \end{bmatrix}. \tag{22}$$

3.2 Hessian Matrix for Newton's Method

The Hessian matrix is the second derivative of the cost function:

$$H(\theta) = \nabla^2 J(\theta). \tag{23}$$

For nonlinear least-squares problems, the Hessian can be written as

$$H(\theta) = \sum_{i=1}^{N_{\text{tr}}} \left[\left(\frac{\partial \hat{y}_i}{\partial \theta} \right) \left(\frac{\partial \hat{y}_i}{\partial \theta} \right)^T - e_i \frac{\partial^2 \hat{y}_i}{\partial \theta^2} \right]. \tag{24}$$

Define the first-derivative vector

$$g_i = \frac{\partial \hat{y}_i}{\partial \theta} = \begin{bmatrix} s_i \\ A c_i t_i \\ A c_i \\ 1 \end{bmatrix}. \tag{25}$$

The second derivatives of the model are:

$$\frac{\partial^2 \hat{y}_i}{\partial A^2} = 0, \tag{26}$$

$$\frac{\partial^2 \hat{y}_i}{\partial A \partial \omega} = c_i t_i, \tag{27}$$

$$\frac{\partial^2 \hat{y}_i}{\partial A \partial \phi} = c_i, \tag{28}$$

$$\frac{\partial^2 \hat{y}_i}{\partial A \partial c} = 0, \tag{29}$$

$$\frac{\partial^2 \hat{y}_i}{\partial \omega^2} = -A s_i t_i^2, \tag{30}$$

$$\frac{\partial^2 \hat{y}_i}{\partial \omega \partial \phi} = -A s_i t_i, \tag{31}$$

$$\frac{\partial^2 \hat{y}_i}{\partial \omega \partial c} = 0, \tag{32}$$

$$\frac{\partial^2 \hat{y}_i}{\partial \phi^2} = -A s_i, \tag{33}$$

$$\frac{\partial^2 \hat{y}_i}{\partial \phi \partial c} = 0, \tag{34}$$

$$\frac{\partial^2 \hat{y}_i}{\partial c^2} = 0. \tag{35}$$

Therefore, the second-derivative matrix of the model is

$$\frac{\partial^2 \hat{y}_i}{\partial \theta^2} = \begin{bmatrix} 0 & c_i t_i & c_i & 0 \\ c_i t_i & -A s_i t_i^2 & -A s_i t_i & 0 \\ c_i & -A s_i t_i & -A s_i & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{36}$$

This Hessian was used in the implementation of Newton's method.

4 Gradient Descent Implementation

Gradient Descent updates the parameter vector according to

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta^{(k)}), \tag{37}$$

where α is the learning rate.

The initial parameter vector was chosen as

$$\theta^{(0)} = \begin{bmatrix} A_0 \\ \omega_0 \\ \phi_0 \\ c_0 \end{bmatrix} = \begin{bmatrix} \frac{\max(y_{\text{train}}) - \min(y_{\text{train}})}{2} \\ 1 \\ 0 \\ \text{mean}(y_{\text{train}}) \end{bmatrix}. \tag{38}$$

The stopping criterion was based on the gradient norm:

$$\|\nabla J(\theta^{(k)})\| < 10^{-6}, \tag{39}$$

or reaching the maximum number of iterations.

The hyperparameters used were:

- learning rate: $\alpha = 10^{-4}$,
- maximum iteration number: 5000,
- tolerance: 10^{-6}.

The method converged after 3369 iterations.

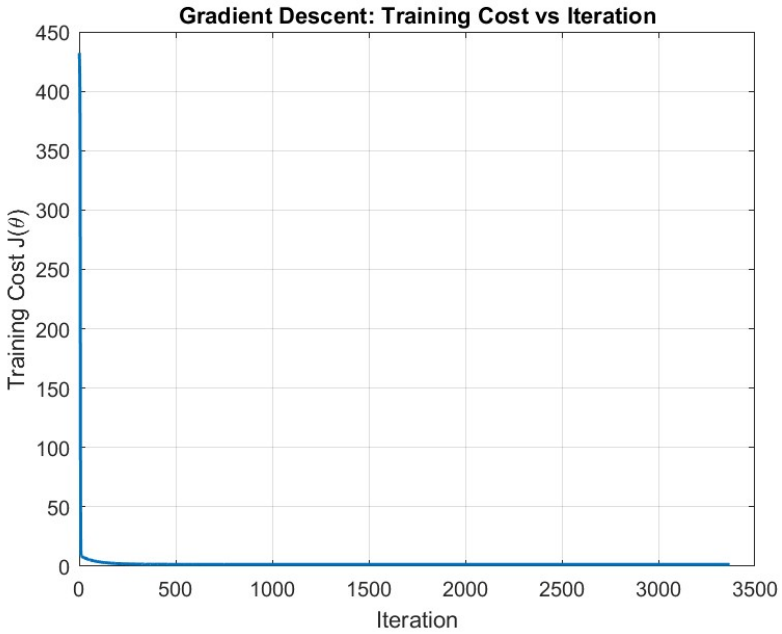


Figure 1: Training cost versus iteration number for standard Gradient Descent.

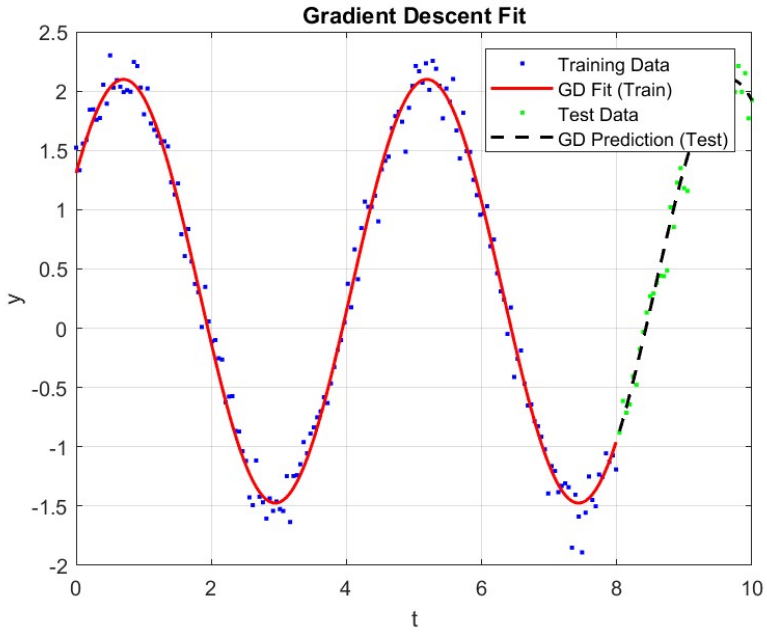


Figure 2: Gradient Descent method function fit for given train and test data.

5 Gradient Descent with Momentum Implementation

Gradient Descent with Momentum improves the convergence of standard Gradient Descent by introducing a velocity term. The update equations are

$$v^{(k+1)} = \beta v^{(k)} - \alpha \nabla J(\theta^{(k)}), \tag{40}$$

$$\theta^{(k+1)} = \theta^{(k)} + v^{(k+1)}, \tag{41}$$

where β is the momentum coefficient.

The same initial parameter vector was used for a fair comparison:

$$\theta^{(0)} = \begin{bmatrix} \dfrac{\max(y_{\text{train}}) - \min(y_{\text{train}})}{2} \\ 1 \\ 0 \\ \text{mean}(y_{\text{train}}) \end{bmatrix}. \tag{42}$$

The initial velocity was set to zero:

$$v^{(0)} = 0. \tag{43}$$

The hyperparameters were:

- learning rate: $\alpha = 10^{-4}$,
- momentum coefficient: $\beta = 0.9$,
- maximum iteration number: 5000,
- tolerance: 10^{-6}.

The stopping criterion was the same as in standard Gradient Descent. The method converged after 362 iterations.

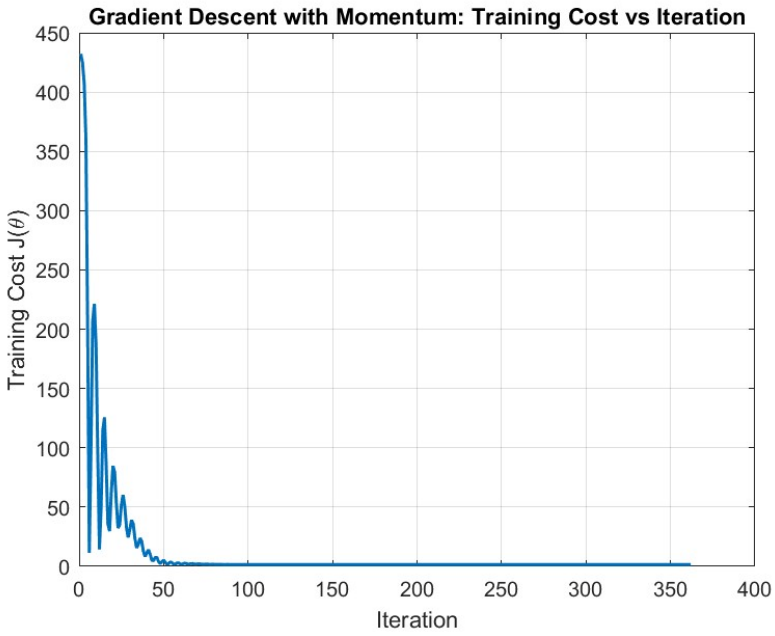


Figure 3: Training cost versus iteration number for Gradient Descent with Momentum.

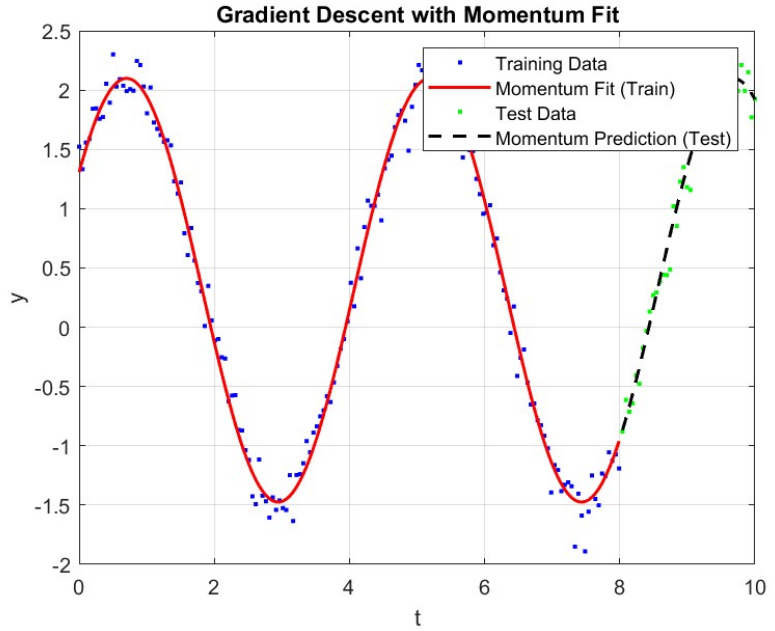


Figure 4: Gradient Descent with Momentum method function fit for given train and test data.

6 Newton's Method Implementation

Newton's method uses second-order information through the Hessian matrix. The update rule is

$$\theta^{(k+1)} = \theta^{(k)} - H^{-1}(\theta^{(k)})\nabla J(\theta^{(k)}). \tag{44}$$

In implementation, instead of computing the inverse explicitly, the Newton step $p^{(k)}$ was obtained by solving

$$H(\theta^{(k)})p^{(k)} = \nabla J(\theta^{(k)}). \tag{45}$$

Then the update becomes

$$\theta^{(k+1)} = \theta^{(k)} - p^{(k)}. \tag{46}$$

To improve numerical stability, a small regularization term was added to the Hessian:

$$H_{\text{reg}} = H + \lambda I, \tag{47}$$

with $\lambda = 10^{-6}$.

For fair comparison, the same initial parameter vector was used:

$$\theta^{(0)} = \begin{bmatrix} \dfrac{\max(y_{\text{train}}) - \min(y_{\text{train}})}{2} \\ 1 \\ 0 \\ \text{mean}(y_{\text{train}}) \end{bmatrix}. \tag{48}$$

The hyperparameters were:

- regularization parameter: $\lambda = 10^{-6}$,
- maximum iteration number: 100,
- tolerance: 10^{-6}.

Newton's method converged after only 8 iterations. However, despite this fast convergence in terms of iteration count, it converged to a poor solution.

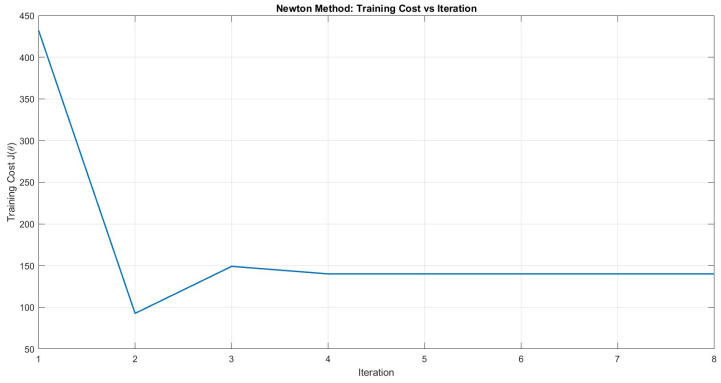


Figure 5: Training cost versus iteration number for Newton's Method.

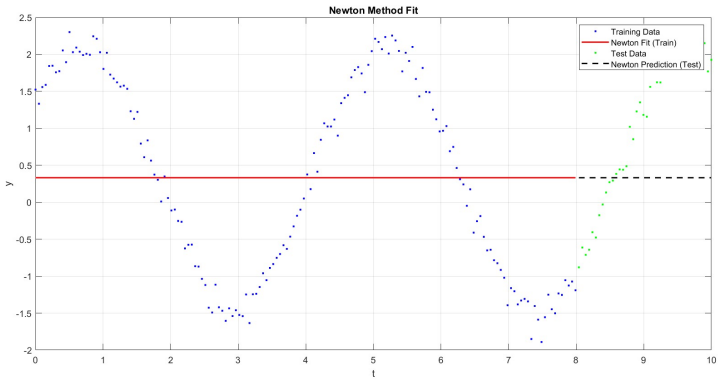


Figure 6: Newton's Method function fit for given train and test data.

7 Comparison of Estimated Parameters and Prediction Accuracy

The estimated parameter values for all three methods are listed in Table 1. The test-set Mean Squared Error (MSE) values are given in Table 2.

Table 1: Estimated parameter values for the three optimization methods.

Method	A	ω	ϕ	c
Gradient Descent	1.7873	1.3998	0.5904	0.3120
Momentum GD	1.7873	1.3998	0.5904	0.3120
Newton's Method	0.0000	0.7262	1.4088	0.3315

Table 2: Convergence iteration count and test-set MSE values.

Method	Iterations to Converge	Test MSE
Gradient Descent	3369	0.016294
Momentum GD	362	0.016294
Newton's Method	8	1.530192

According to the results, standard Gradient Descent and Momentum Gradient Descent converged to the same parameter vector and produced exactly the same test-set performance. However, Momentum GD reached the solution much faster, requiring only 362 iterations compared to 3369 iterations for standard GD.

On the other hand, Newton's method converged in only 8 iterations but produced a very poor parameter estimate. In particular, the estimated amplitude became nearly zero, which resulted in an almost flat fitted signal and a very large test MSE. Therefore, the fastest method in terms of iteration count did not provide the best parameter estimation or the best generalization performance.

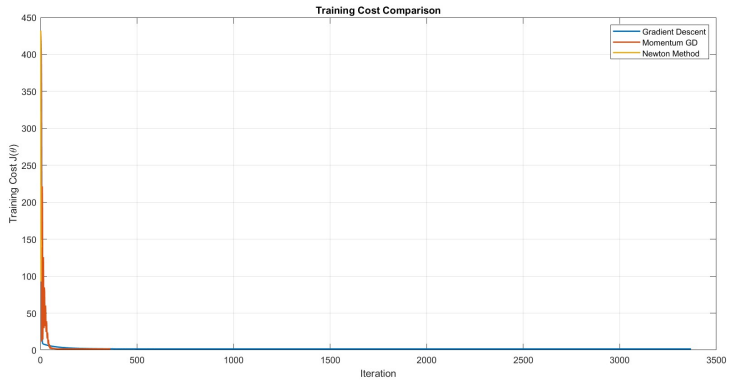


Figure 7: Comparison of the training cost histories of Gradient Descent, Momentum Gradient Descent, and Newton's Method.

8 Discussion and Interpretation

8.1 Convergence Behavior

The three methods showed clearly different convergence behaviors:

- Standard Gradient Descent converged reliably, but very slowly.
- Momentum Gradient Descent converged to the same solution much faster.
- Newton's Method converged in very few iterations, but to a poor solution.

This shows that a smaller number of iterations does not necessarily imply a better final estimate.

8.2 Sensitivity to the Initial Guess

This nonlinear regression problem is nonconvex because of the sinusoidal model structure. Therefore, all methods are expected to be sensitive to the initial guess to some extent. However, Newton's method was observed to be much more sensitive than the first-order methods. Even though the same initial point was used for all methods, Newton's method was driven to an undesirable solution. This indicates that the local curvature information encoded in the Hessian may not always guide the optimizer toward the globally meaningful minimum.

8.3 Training Cost and Test Performance

In this problem, lower training cost generally corresponded to better test performance for Gradient Descent and Momentum GD. However, Newton's method demonstrated that fast optimization progress alone is not sufficient. Since the optimization landscape is nonlinear and nonconvex, a method may converge rapidly to a poor stationary point. Therefore, lower iteration count or fast convergence does not automatically guarantee better generalization on the test set.

8.4 Interpretation of the Results

The results suggest that Momentum Gradient Descent provided the best overall trade-off between accuracy and convergence speed. It achieved the same parameter estimate and the same test performance as standard Gradient Descent, while requiring far fewer iterations. Newton's method, although theoretically attractive due to its second-order update, was not robust in this experiment. This behavior is likely due to the nonconvexity of the cost surface and the sensitivity of the Hessian-based update to the local geometry around the initial guess.

9 Conclusion

In this study, a nonlinear sinusoidal regression problem was formulated and solved using three different optimization methods. The gradient and Hessian of the cost function were derived analytically, and all methods were implemented using the same initialization for fair comparison.

The results showed that:

- Gradient Descent successfully estimated the parameters but required many iterations.
- Gradient Descent with Momentum achieved the same solution much faster.
- Newton's Method converged very quickly in terms of iteration count but failed to produce an accurate model.

Therefore, among the tested methods, Gradient Descent with Momentum gave the best overall performance in this problem.

10 Question 2: Regulation via Optimization

In this question, an optimization-based regulation strategy was designed for the discrete-time linear system

$$x_{k+1} = Ax_k + Bu_k, \tag{49}$$

where

$$A = \begin{bmatrix} 1 & 0.2 \\ 0 & 0.9 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 0.2 \end{bmatrix}, \qquad x_0 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}. \tag{50}$$

The prediction horizon was chosen as

$$H = 5. \tag{51}$$

At each time step k, a future control sequence

$$U_k = \begin{bmatrix} u_k & u_{k+1} & \cdots & u_{k+H-1} \end{bmatrix}^T \tag{52}$$

was optimized by minimizing a finite-horizon quadratic cost function. Only the first control action of the optimized sequence was applied to the system, and the optimization was repeated at the next time step. This corresponds to a receding-horizon optimization-based control strategy.

10.1 Prediction Model Derivation

Starting from

$$x_{k+1} = Ax_k + Bu_k, \tag{53}$$

the predicted future states can be derived recursively.

For one step ahead,

$$x_{k+1} = Ax_k + Bu_k. \tag{54}$$

For two steps ahead,

$$\begin{aligned} x_{k+2} &= Ax_{k+1} + Bu_{k+1} && (55)\\ &= A(Ax_k + Bu_k) + Bu_{k+1} && (56)\\ &= A^2x_k + ABu_k + Bu_{k+1}. && (57) \end{aligned}$$

For three steps ahead,

$$\begin{aligned} x_{k+3} &= Ax_{k+2} + Bu_{k+2} && (58)\\ &= A^3x_k + A^2Bu_k + ABu_{k+1} + Bu_{k+2}. && (59) \end{aligned}$$

Therefore, at a general prediction step j, the predicted state is

$$x_{k+j} = A^j x_k + \sum_{m=0}^{j-1} A^{j-1-m} B u_{k+m}, \qquad j = 1, 2, \ldots, H. \tag{60}$$

This expression shows that each future state depends on the current state x_k and on the future control sequence U_k.

It is also convenient to write the full prediction model in compact stacked form. Define

$$X_k = \begin{bmatrix} x_{k+1} \\ x_{k+2} \\ \vdots \\ x_{k+H} \end{bmatrix}. \tag{61}$$

Then the prediction model can be written as

$$X_k = \mathcal{A}x_k + \mathcal{B}U_k, \tag{62}$$

where

$$\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^H \end{bmatrix}, \tag{63}$$

and

$$\mathcal{B} = \begin{bmatrix} B & 0 & 0 & \cdots & 0 \\ AB & B & 0 & \cdots & 0 \\ A^2B & AB & B & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A^{H-1}B & A^{H-2}B & A^{H-3}B & \cdots & B \end{bmatrix}. \tag{64}$$

10.2 Cost-Function Formulation

The finite-horizon cost function at time step k is defined as

$$J_k(U_k) = \sum_{i=0}^{H-1} \left(x_{k+i}^T Q x_{k+i} + r u_{k+i}^2 \right) + x_{k+H}^T Q_f x_{k+H}, \tag{65}$$

where

$$Q = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}, \qquad Q_f = \begin{bmatrix} 20 & 0 \\ 0 & 2 \end{bmatrix}, \qquad r = 0.2. \tag{66}$$

The role of these weighting terms is as follows:

- Q penalizes the predicted state deviation from the origin at intermediate steps,
- Q_f penalizes the final predicted state more strongly at the end of the horizon,
- r penalizes control effort.

Since the state vector is

$$x_k = \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix}, \tag{67}$$

the state penalty term becomes

$$x_{k+i}^T Q x_{k+i} = 10 x_1^2(k+i) + x_2^2(k+i), \tag{68}$$

and the terminal penalty becomes

$$x_{k+H}^T Q_f x_{k+H} = 20 x_1^2(k+H) + 2 x_2^2(k+H). \tag{69}$$

Therefore, the cost can be written explicitly as

$$J_k(U_k) = \sum_{i=0}^{H-1} \left[10 x_1^2(k+i) + x_2^2(k+i) + r u_{k+i}^2 \right] + 20 x_1^2(k+H) + 2 x_2^2(k+H). \tag{70}$$

10.3 Iterative Optimization-Based Control Algorithm

At each time step, the following procedure was used:

1. Read the current state x_k.
2. Construct the future control sequence $U_k = [u_k, u_{k+1}, \ldots, u_{k+H-1}]^T$.
3. Predict future states over the horizon using the system model.
4. Form the cost function $J_k(U_k)$.
5. Solve the optimization problem

$$U_k^\star = \arg\min_{U_k} J_k(U_k). \tag{71}$$

6. Apply only the first control input

$$u_k = u_k^\star. \tag{72}$$

7. Update the system state using

$$x_{k+1} = Ax_k + Bu_k. \tag{73}$$

8. Repeat the same procedure at the next time step.

This algorithm is essentially a finite-horizon receding-horizon control law.

For implementation, MATLAB's built-in optimization routine `fminunc` was used. The decision variable was the control sequence $U_k \in \mathbb{R}^5$. The solver was executed inside the control loop for 15 time steps. A quasi-Newton algorithm was used, with the zero vector as the initial guess at the first step, and warm-starting was used afterward by shifting the previously optimized sequence.

10.4 Implementation and Simulation for $r = 0.2$

The controller was simulated for 15 time steps using the nominal value

$$r = 0.2. \tag{74}$$

The resulting state trajectories are shown in Figure 8, and the applied control sequence is shown in Figure 9.

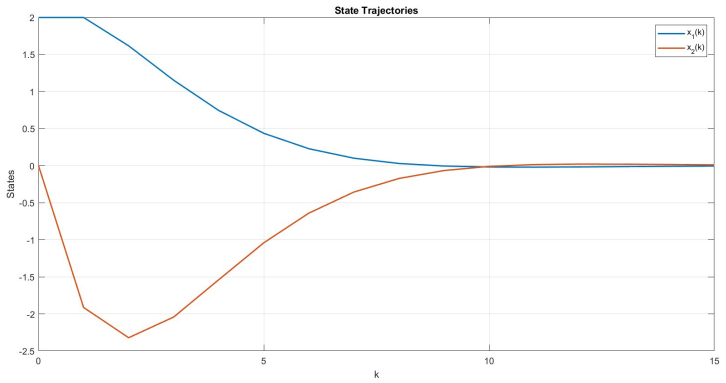


Figure 8: State trajectories $x_1(k)$ and $x_2(k)$ for the optimization-based controller with $r = 0.2$.

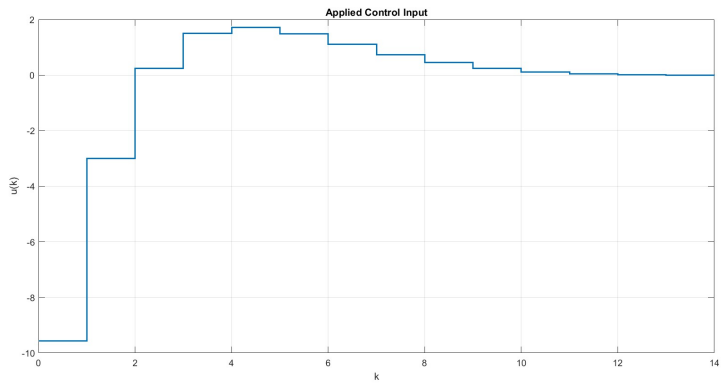


Figure 9: Applied control input $u(k)$ for the optimization-based controller with $r = 0.2$.

From the obtained trajectories, it is observed that both state variables are driven toward the origin successfully. The first state $x_1(k)$ decreases gradually from its initial value of 2 and approaches zero smoothly. The second state $x_2(k)$ initially becomes negative, reaches a minimum, and then recovers toward zero. This transient behavior is expected because the controller first generates a strong corrective action to move the state toward the origin over the finite prediction horizon.

The control input is initially large and negative, then changes sign and gradually decays toward zero. This behavior indicates that the optimizer is actively using the control authority to regulate the system while balancing state regulation and control effort according to the prescribed cost function.

10.5 Effect of the Control-Effort Penalty

To analyze the effect of the control-effort penalty, the same simulation was repeated for the following two additional cases:

$$r = 0.05, \qquad r = 1.0. \tag{75}$$

The expected effect of changing r is as follows:

- When r is smaller, control effort is penalized less, so the controller becomes more aggressive. Larger control actions are expected, and the states are generally driven to the origin more rapidly.
- When r is larger, control effort is penalized more heavily, so the controller becomes more conservative. Smaller control actions are expected, and the states are generally regulated more slowly.

Case 1: $r = 0.05$

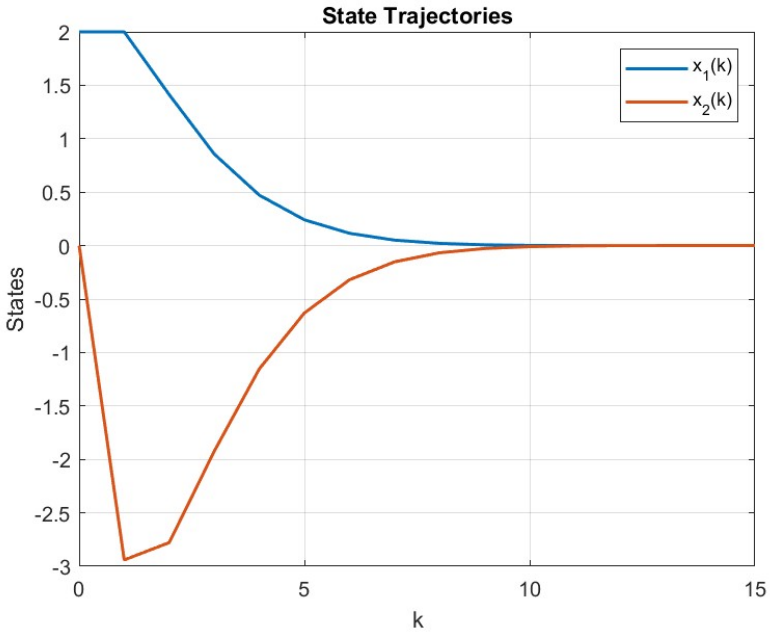


Figure 10: State trajectories for $r = 0.05$.

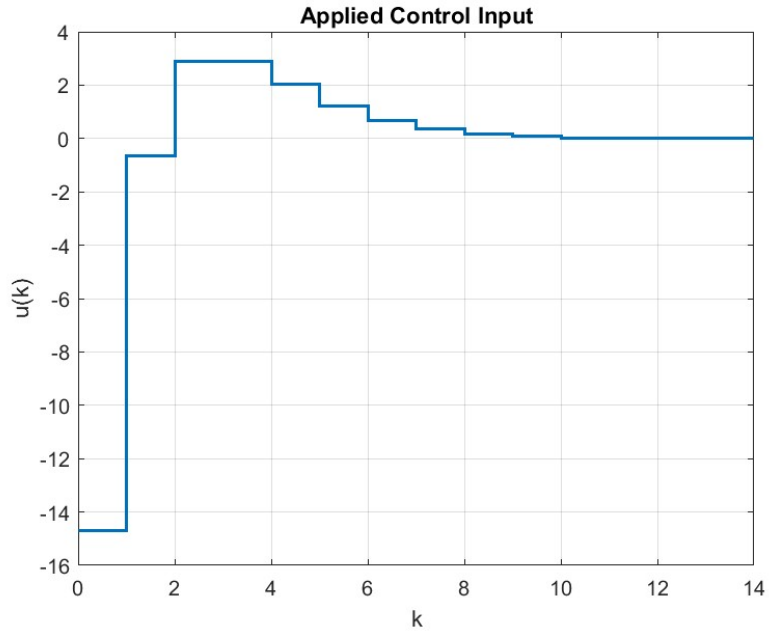


Figure 11: Applied control input for $r = 0.05$.

For $r = 0.05$, the controller is expected to be more aggressive than in the nominal case $r = 0.2$. Consequently, the magnitude of the control input is larger, and the state trajectories approach the origin more rapidly.

Case 2: $r = 1.0$

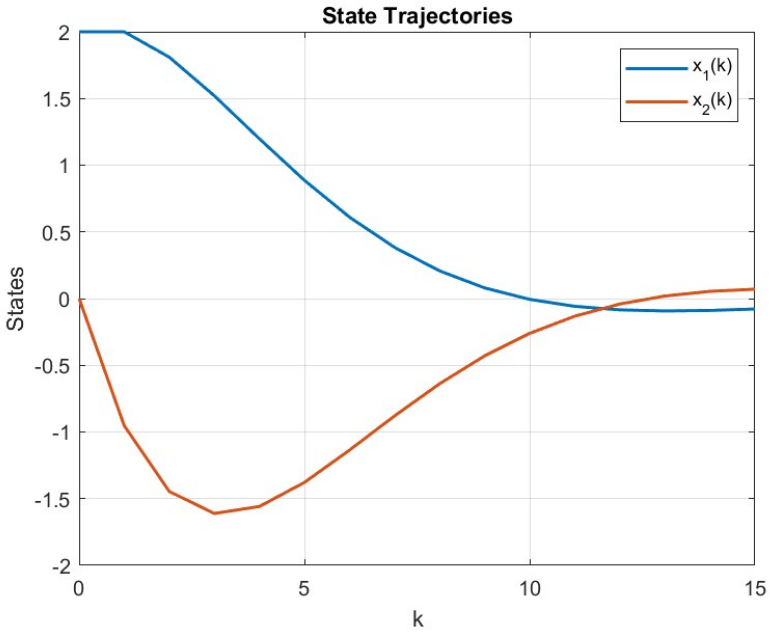


Figure 12: State trajectories for $r = 1.0$.

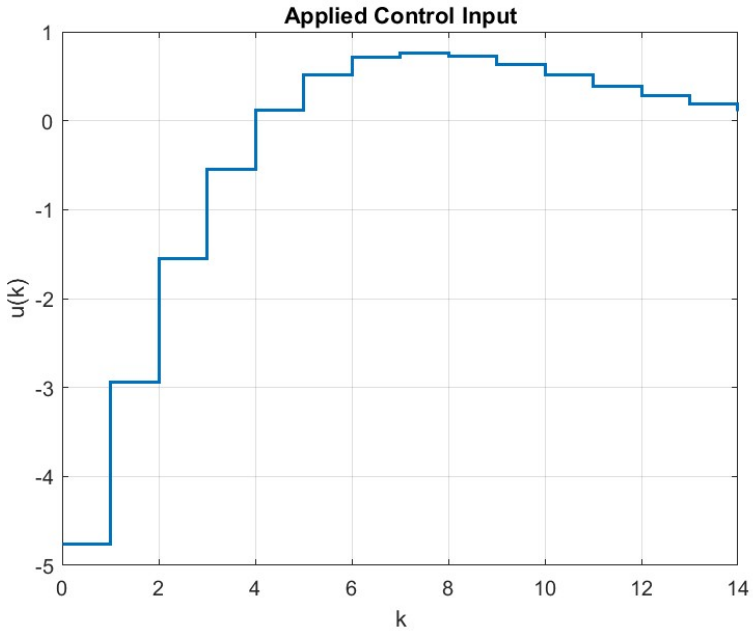


Figure 13: Applied control input for $r = 1.0$.

For $r = 1.0$, the controller is expected to be more conservative than in the nominal case $r = 0.2$. Therefore, the control input magnitude is smaller, and the state trajectories approach the origin more slowly.

10.6 Discussion on Optimization Behavior

The optimization process inside the control loop converged reliably in the nominal simulation. At each time step, the solver returned an optimized future input sequence, and the applied first control input successfully regulated the system toward the origin.

The use of warm-starting was beneficial because the previously optimized control sequence provides a reasonable initial guess for the next optimization problem. Since successive optimization problems are similar in a receding-horizon framework, this generally improves convergence speed and reliability.

In this problem, the optimization was not highly sensitive to the initial guess because the cost function is quadratic in the state and input, and the system is linear. Therefore, the solver behaved robustly and produced consistent control actions throughout the simulation.

10.7 Conclusion

An optimization-based regulation strategy was implemented for a discrete-time linear system using a receding-horizon framework with prediction horizon $H = 5$. The future state evolution was derived explicitly in terms of the current state and the future control sequence, and a quadratic finite-horizon cost function was minimized at each time step using MATLAB's `fminunc` solver.

For the nominal case $r = 0.2$, the controller successfully drove the system states toward the origin. The control input exhibited an initially strong corrective action and then decayed gradually toward zero. This demonstrates that the optimization-based controller can regulate the system effectively.

The effect of the control-effort penalty is conceptually clear: decreasing r makes the controller more aggressive, whereas increasing r makes it more conservative. After inserting the additional simulation results for $r = 0.05$ and $r = 1.0$, the qualitative and quantitative comparison can be completed directly.

11 Question 3: Adaptive PID

In this question, a fixed PID controller and an adaptive PID-like controller were designed and compared for the nonlinear discrete-time plant

$$y(k+1) = \frac{y(k)}{1 + 0.1y^2(k)} + u(k), \qquad y(0) = 0. \tag{76}$$

The reference signal is defined as

$$r(k) = \begin{cases} 0.5, & 1 \le k \le 20, \\ 1.0, & 21 \le k \le 50, \\ 0.8, & 51 \le k \le N, \end{cases} \qquad N = 80. \tag{77}$$

The tracking error is

$$e(k) = r(k) - y(k). \tag{78}$$

The main objective is to compare the two controllers in terms of tracking performance and adaptive behavior.

11.1 Controller Formulation

11.1.1 Fixed PID Controller

The fixed PID control law is given by

$$u(k) = K_P e(k) + K_I \sum_{j=0}^{k} e(j) + K_D\Big(e(k) - e(k-1)\Big). \tag{79}$$

For implementation, it is convenient to define

$$x_P(k) = e(k), \tag{80}$$

$$x_I(k) = x_I(k-1) + e(k), \tag{81}$$

$$x_D(k) = e(k) - e(k-1), \tag{82}$$

so that the control signal can be written as

$$u(k) = K_P x_P(k) + K_I x_I(k) + K_D x_D(k). \tag{83}$$

The roles of the three terms are as follows:

- The proportional term reacts to the instantaneous tracking error.
- The integral term accumulates past error and helps reduce steady-state offset.
- The derivative term reacts to the rate of change of the error and improves damping.

11.1.2 Adaptive PID-like Controller

For the adaptive controller, the PID-like feature channels are defined as

$$x_P(k) = e(k), \tag{84}$$

$$x_I(k) = x_I(k-1) + e(k), \tag{85}$$

$$x_D(k) = e(k) - e(k-1), \tag{86}$$

with the initial conditions

$$e(0) = 0, \qquad x_I(0) = 0. \tag{87}$$

The adaptive control signal is

$$u(k) = w_P(k)x_P(k) + w_I(k)x_I(k) + w_D(k)x_D(k). \tag{88}$$

The adaptive weights are updated according to

$$w_i(k+1) = w_i(k) + \eta \, \frac{e(k)x_i(k)}{1 + x_P^2(k) + x_I^2(k) + x_D^2(k)}, \qquad i \in \{P, I, D\}, \tag{89}$$

where

$$\eta = 0.03. \tag{90}$$

The initial weights were given as

$$w_P(0) = 0.8, \qquad w_I(0) = 0.05, \qquad w_D(0) = 0.10. \tag{91}$$

After each update, the weights were clipped to satisfy

$$0 \le w_P(k) \le 3.0, \tag{92}$$

$$0 \le w_I(k) \le 1.0, \tag{93}$$

$$0 \le w_D(k) \le 1.0. \tag{94}$$

This controller is called *PID-like* because it preserves the proportional, integral, and derivative channel structure, but the coefficients multiplying these channels are no longer fixed constants. Instead, they are adapted online according to the tracking performance.

11.2 Simulation of the Fixed PID Controller

For the fixed PID controller, the following gains were selected:

$$K_P = 1.0, \qquad K_I = 0.05, \qquad K_D = 0.10. \tag{95}$$

These values were chosen to obtain a good compromise between fast tracking, acceptable overshoot, and stable response for the nonlinear plant. In particular:

- $K_P = 1.0$ provides a sufficiently strong reaction to the tracking error,
- $K_I = 0.05$ is kept moderate in order to reduce steady-state error without causing excessive oscillation,
- $K_D = 0.10$ helps damp the transient response near the step changes in the reference.

The resulting output tracking response is shown in Figure 14, the control signal is shown in Figure 15, and the tracking error is shown in Figure 16.

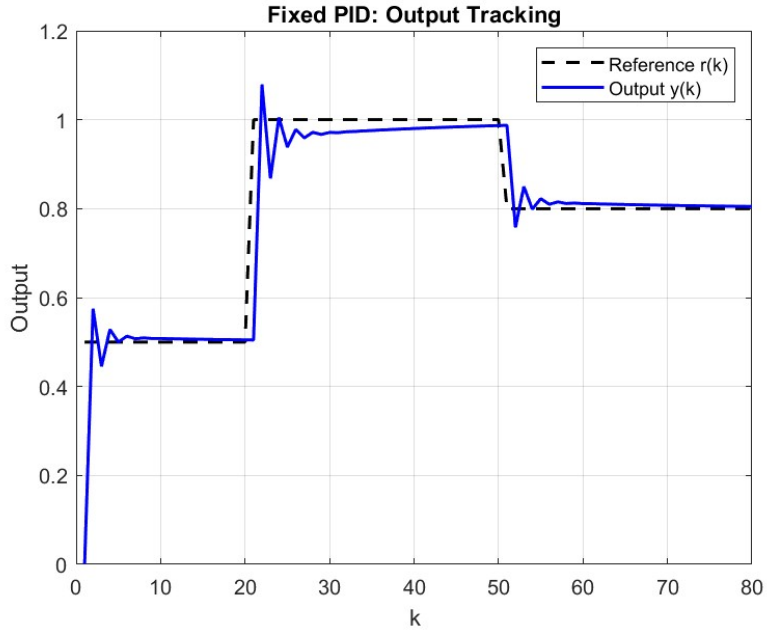


Figure 14: Output tracking of the fixed PID controller.

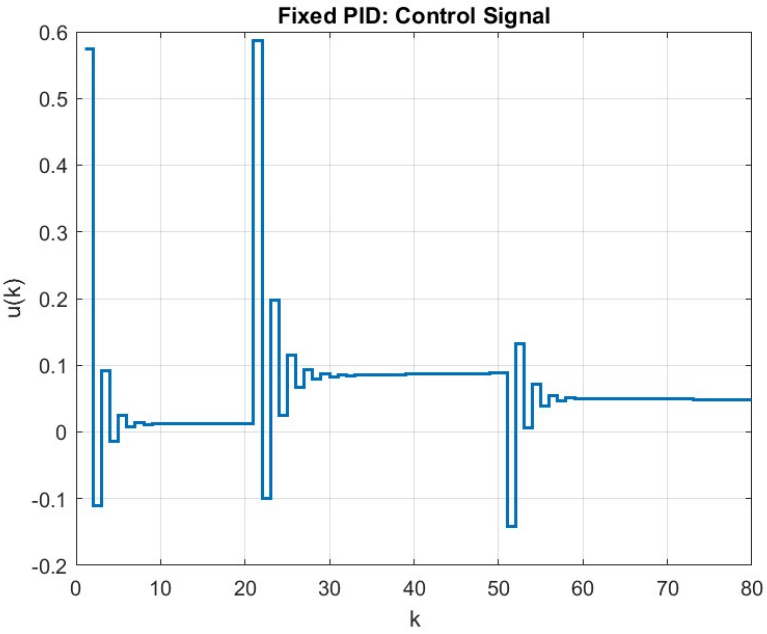


Figure 15: Control signal of the fixed PID controller.

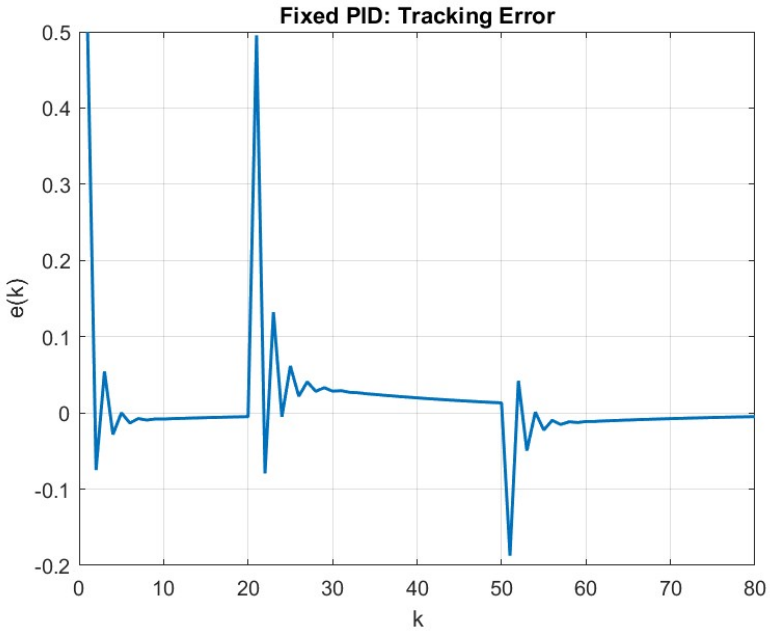


Figure 16: Tracking error of the fixed PID controller.

The fixed PID controller tracks all three reference levels successfully. The response shows small transient oscillations immediately after the reference changes, especially around the transition from 0.5 to 1.0, but the output remains well regulated and settles near the desired values. The control input exhibits short transient peaks at the reference transitions and then decreases to smaller steady values.

11.3 Simulation of the Adaptive PID-like Controller

The same nonlinear plant was then simulated using the adaptive PID-like controller with the prescribed adaptation rule and initial weights. The resulting output tracking response is shown in Figure 17, the control signal in Figure 18, the tracking error in Figure 19, and the adaptive weight evolution in Figure 20.

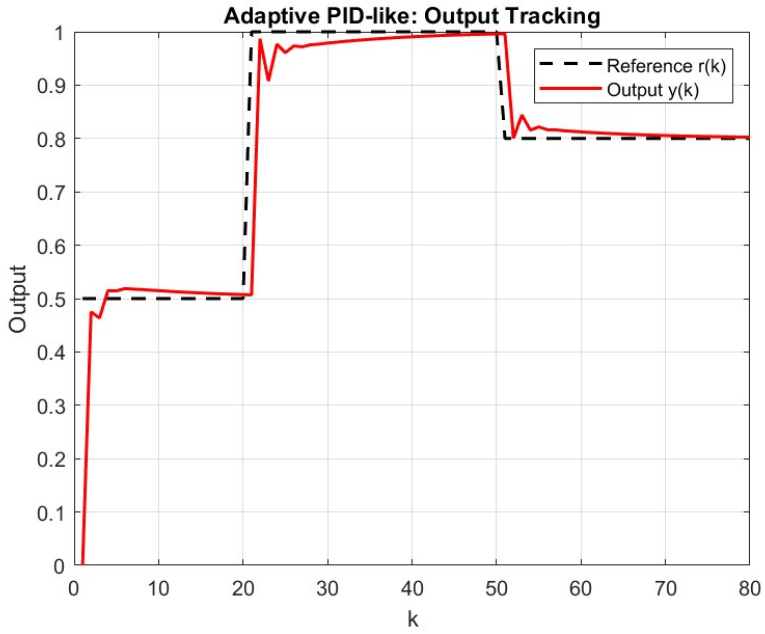


Figure 17: Output tracking of the adaptive PID-like controller.

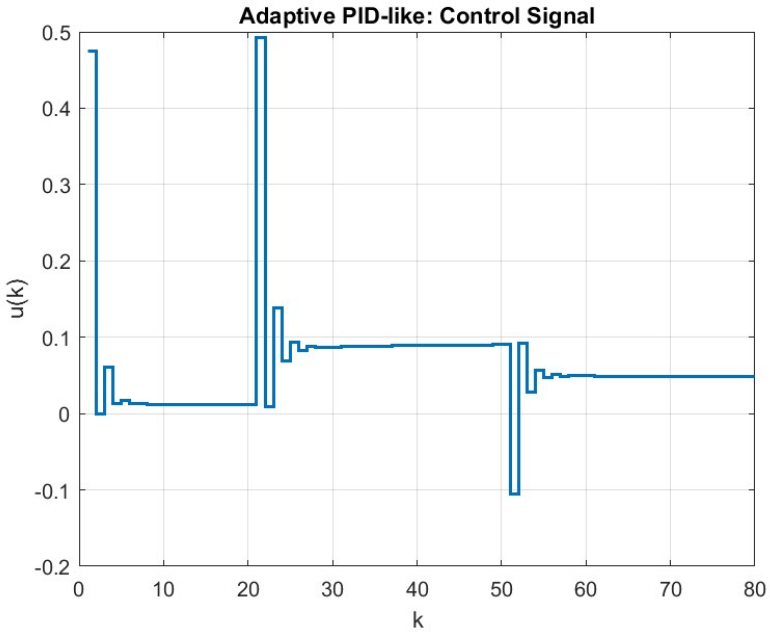


Figure 18: Control signal of the adaptive PID-like controller.

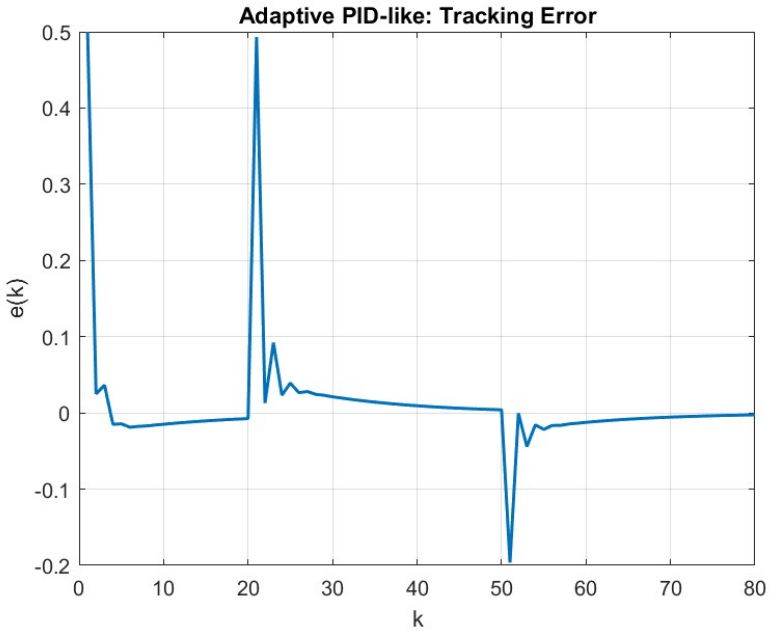


Figure 19: Tracking error of the adaptive PID-like controller.

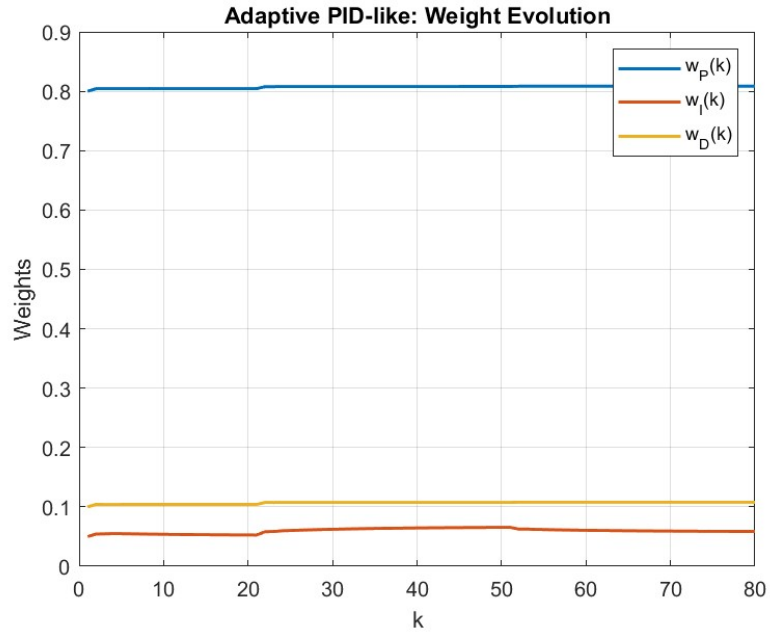


Figure 20: Evolution of the adaptive weights $w_P(k)$, $w_I(k)$, and $w_D(k)$.

The adaptive PID-like controller also tracks the changing reference successfully. The output follows the reference levels with small transient deviations and mild oscillations after the step changes. The control signal remains bounded and shows corrective transients at the switching instants. Compared with the fixed PID case, the adaptive controller adjusts its gains online and slightly improves the error measures.

11.4 Performance Comparison

The two controllers were compared using the following performance indices:

$$IAE = \sum_{k=1}^{N} |e(k)|, \tag{96}$$

$$ISE = \sum_{k=1}^{N} e^2(k). \tag{97}$$

The obtained results are listed in Table 3.

Table 3: Performance comparison of fixed PID and adaptive PID-like controllers.

Controller	IAE	ISE
Fixed PID	2.604389	0.588215
Adaptive PID-like	2.231674	0.557373

According to Table 3, the adaptive PID-like controller achieved lower values of both IAE and ISE. Therefore, in this simulation, the adaptive controller provided better overall tracking performance than the fixed PID controller.

11.5 Discussion of Adaptive Behavior

The weight evolution shown in Figure 20 indicates that the adaptive gains change only moderately during the simulation. This means that the initial weights were already reasonable, and only small adjustments were required to improve tracking.

From the plots, the following observations can be made:

- $w_P(k)$ remains the dominant weight throughout the simulation and changes only slightly around its initial value.
- $w_I(k)$ increases gradually and then slightly decreases after the final reference change, indicating that the integral contribution is adapted according to the accumulated error behavior.
- $w_D(k)$ also changes only slightly and remains bounded within its clipping interval.

Among the three weights, the most visible variation is observed in the integral-related weight $w_I(k)$, while $w_P(k)$ stays dominant in magnitude. This is reasonable because the integral channel is directly related to eliminating accumulated tracking offset over longer intervals.

The adaptive controller responds to reference changes by updating the channel weights according to the normalized adaptation law. When a step change occurs, the error magnitude increases, which immediately affects the update term

$$\frac{e(k)x_i(k)}{1 + x_P^2(k) + x_I^2(k) + x_D^2(k)}.$$

As a result, the controller slightly modifies the contribution of the proportional, integral, and derivative channels in order to improve subsequent tracking.

11.6 Interpretation

This controller is called a PID-like adaptive controller because it preserves the classical PID structure through the three feature channels $x_P(k)$, $x_I(k)$, and $x_D(k)$, but replaces the fixed gains by adaptive weights $w_P(k)$, $w_I(k)$, and $w_D(k)$. Hence, the controller maintains the intuitive proportional-integral-derivative decomposition while allowing online gain adjustment.

The adaptive update rule modifies the control behavior online by increasing or decreasing the weights according to the current error and the activity of each PID-like channel. In this way, the controller can react to changing operating conditions and reference transitions without requiring manually retuned fixed gains.

11.7 Conclusion

In this part, a fixed PID controller and an adaptive PID-like controller were implemented for a nonlinear discrete-time plant with piecewise-constant reference tracking. Both controllers were able to track the reference successfully, with acceptable transient behavior and bounded control signals.

The fixed PID controller with

$$K_P = 1.0, \qquad K_I = 0.05, \qquad K_D = 0.10$$

provided satisfactory performance. However, the adaptive PID-like controller achieved lower error indices:

$$IAE_{\text{adaptive}} < IAE_{\text{fixed}}, \qquad ISE_{\text{adaptive}} < ISE_{\text{fixed}}.$$

Therefore, the adaptive PID-like controller performed slightly better overall in this experiment. The improvement is explained by the online adaptation of the PID-like channel weights, which allows the controller to respond more flexibly to the nonlinear plant dynamics and to the changes in the reference signal.